

Hate Speech Detection

Data Science Intern at Data Glacier

Name: Mehedi Hasan Shakil

Email: shakilcuetcse@gmail.com

Country: Bangladesh

Date: 30-June-2024



Your Deep Learning Partner

Outline

- **Problem Statement**
- **Data Information**
- **Data Preprocessing**
- **Exploratory Data Analysis (EDA)**
- **Feature Extraction**
- **Model Building**
- **Model Performance Evaluation**
- **Application Deployment**
- **Conclusion**



Problem Statement

- ❑ Hate speech targets identity traits like race, religion, or gender.
- ❑ Twitter is a major platform for spreading such content.
- ❑ Manual moderation is time-consuming and inefficient.
- ❑ Build an ML model to detect hate speech in tweets.



Name of dataset	hate_speech.csv
Total number of observations	31962
Total number of features	3
Base format of the file	.csv
Size of the Data	2.95 MB

The dataset has three features:

- id: data type int64
- label: data type int64
- tweet: data type object

There is no missing value in the dataset.

Data Information

Data Preprocessing

- **Text Normalization:** Convert all text to lowercase, remove usernames (e.g., @user), URLs, special characters, and trim unnecessary spaces.
- **Stopword Elimination:** Filter out common words (like “the”, “and”) that don’t add useful meaning.
- **Tokenization:** Break down each sentence into individual word tokens.
- **Lemmatization:** Transform words to their root form (e.g., “running” → “run”) for uniformity.

Original Tweet:

@User123 Running faster than ever! Visit
http://fasttrack.com #speed

After Preprocessing:

1. **Lowercase + Remove Mentions/URLs/Special Chars**
"running faster than ever visit speed"
2. **Remove Stopwords**
"running faster ever visit speed"
3. **Tokenization**
["running", "faster", "ever", "visit", "speed"]
4. **Lemmatization**
["run", "fast", "ever", "visit", "speed"]

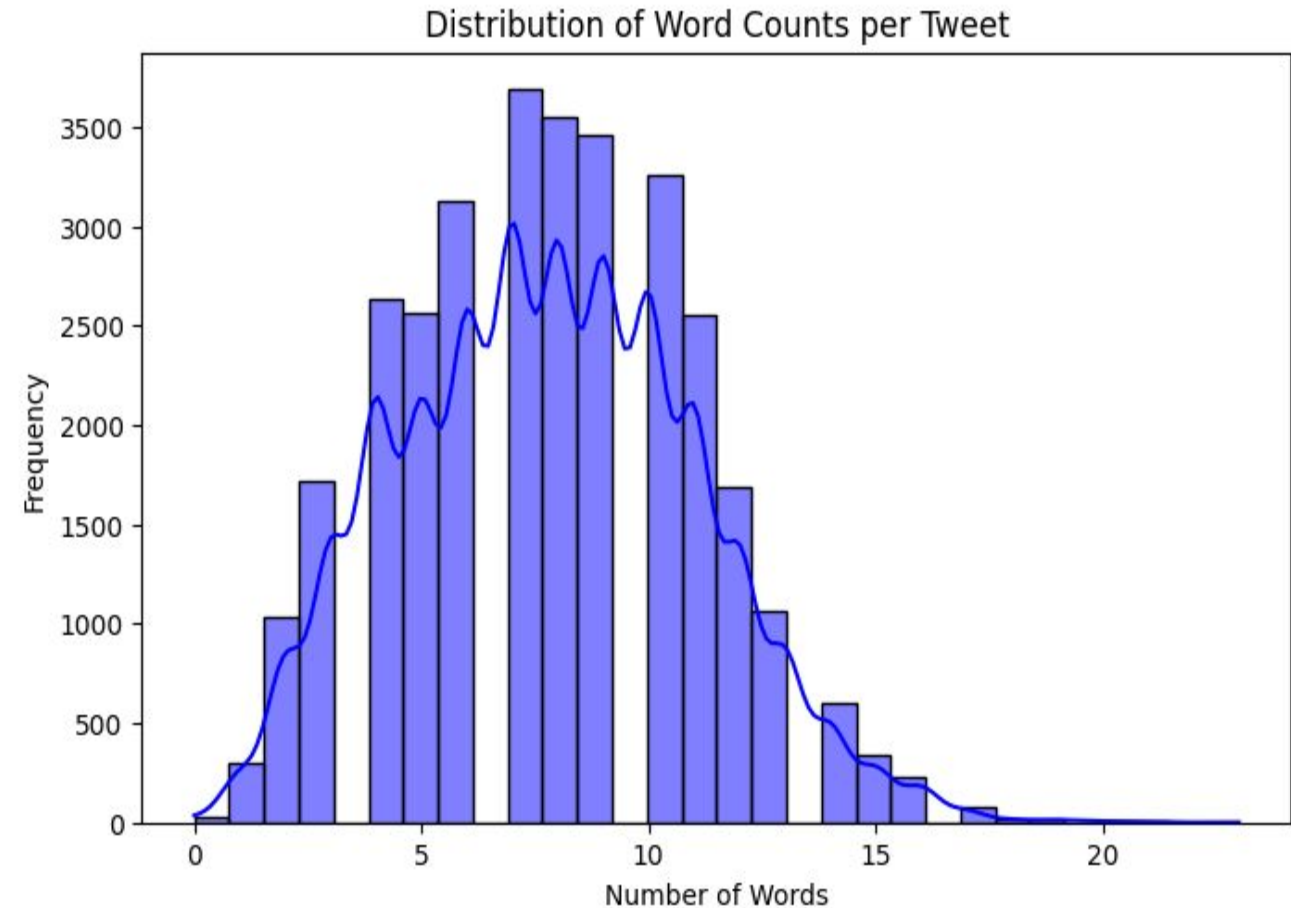
EDA

1. Number of Words per Tweet
2. Most Frequent Words in the Whole Dataset
3. Most Frequent Words in Hate vs. Non-Hate Tweets
4. Word Cloud for Each Level
5. Count of Hate Speech vs. Non-Hate Speech
6. Solve the Imbalance Issue

EDA

Number of Words per Tweet

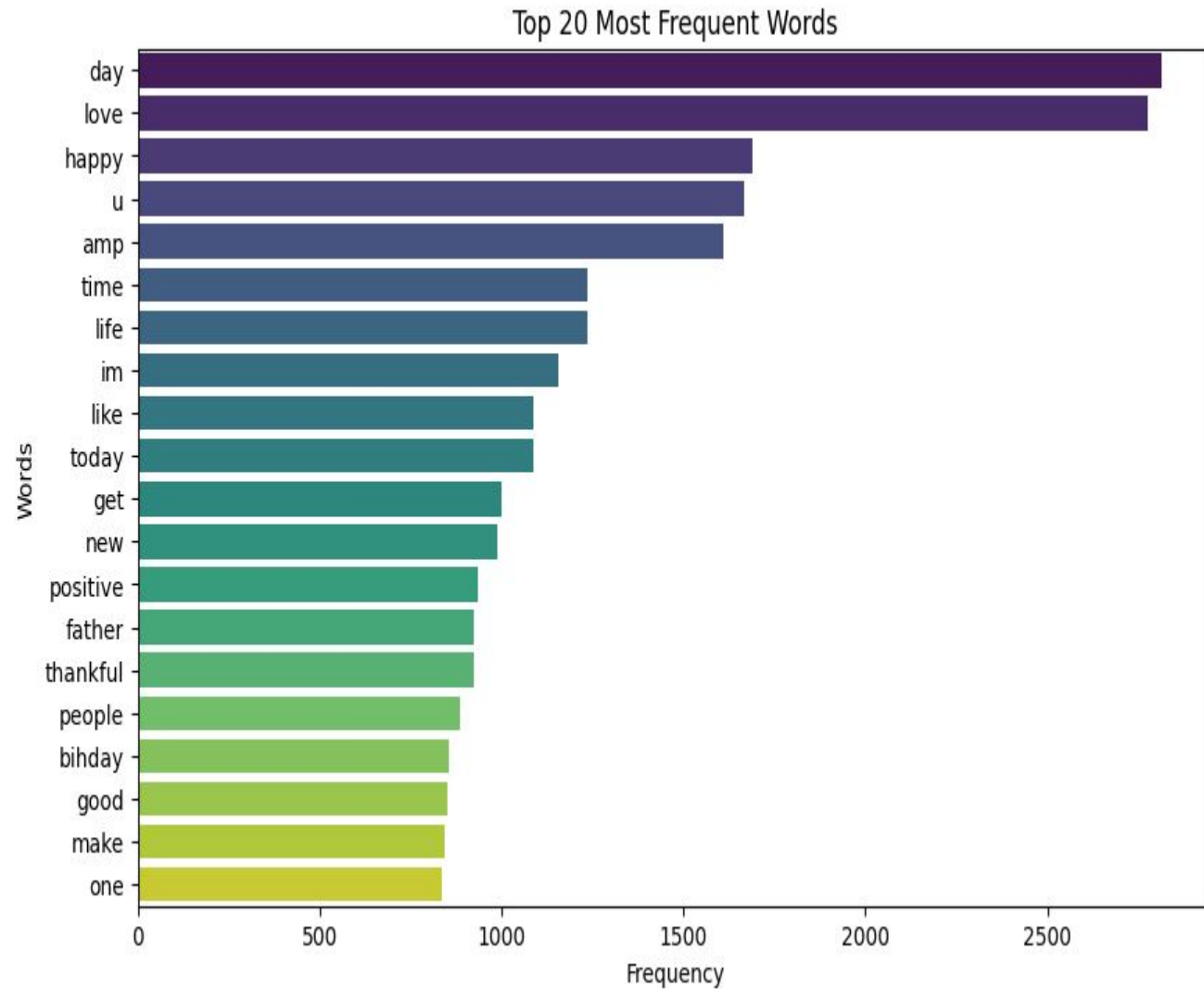
- The majority of tweets contain between 5 to 15 words.



EDA

Most Frequent Words in the Whole Dataset

- The 20 most frequently occurring words across the entire dataset.

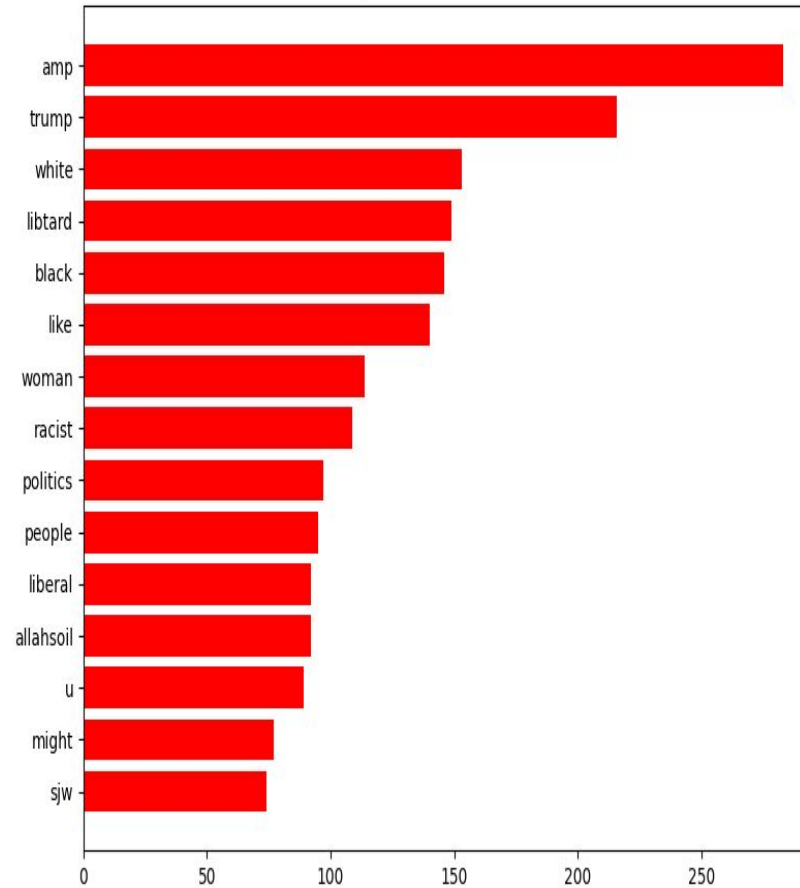


EDA

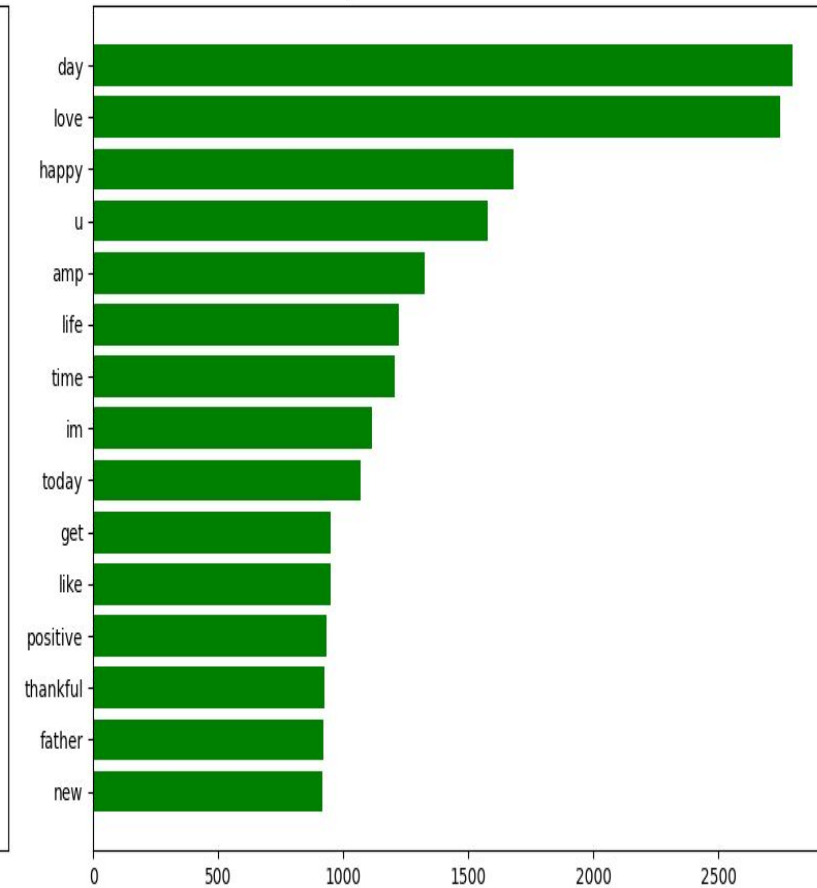
Most Frequent Words in Hate vs. Non-Hate Tweets

- Hate tweets tend to use more aggressive or targeted language, while non-hate tweets exhibit more neutral or conversational vocabulary.

Top Words in Hate Tweets



Top Words in Non-Hate Tweets



Word Cloud for Each Level

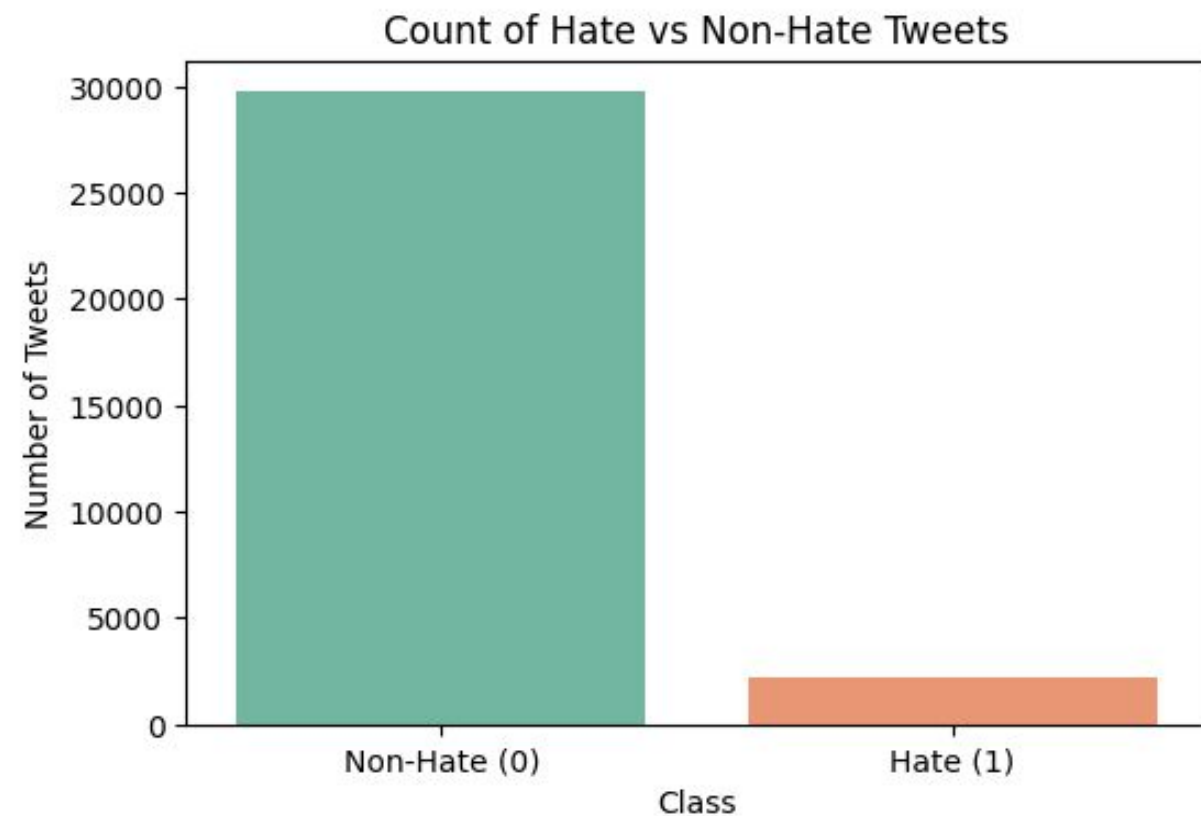
- [illegible]

[illegible]

EDA

Count of Hate Speech vs. Non-Hate Speech

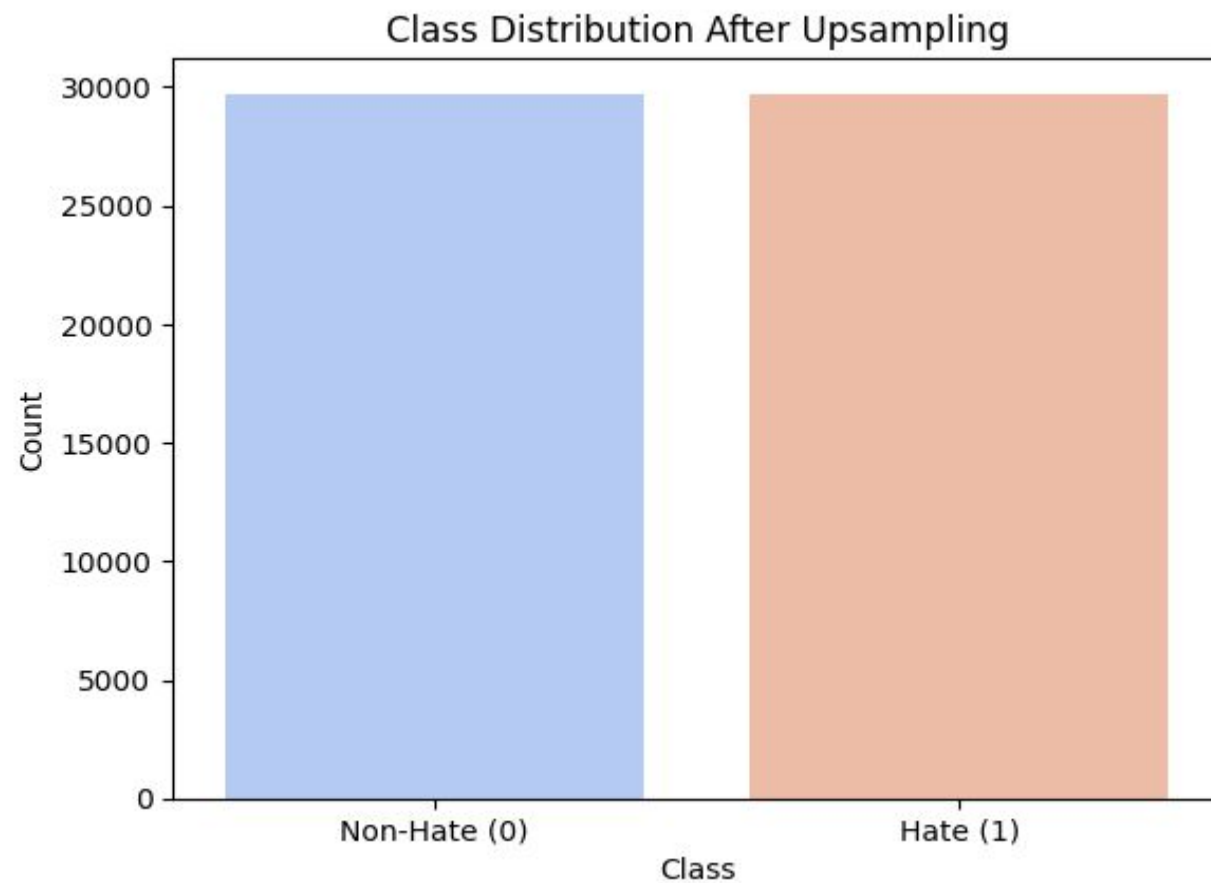
- The original class distribution is visualized.



EDA

Solve the Imbalance Issue

- Applied upsampling to the minority class (hate tweets).



- Bag of Words (BoW)
- TF-IDF
- Word Embeddings (Word2Vec)
- BERT

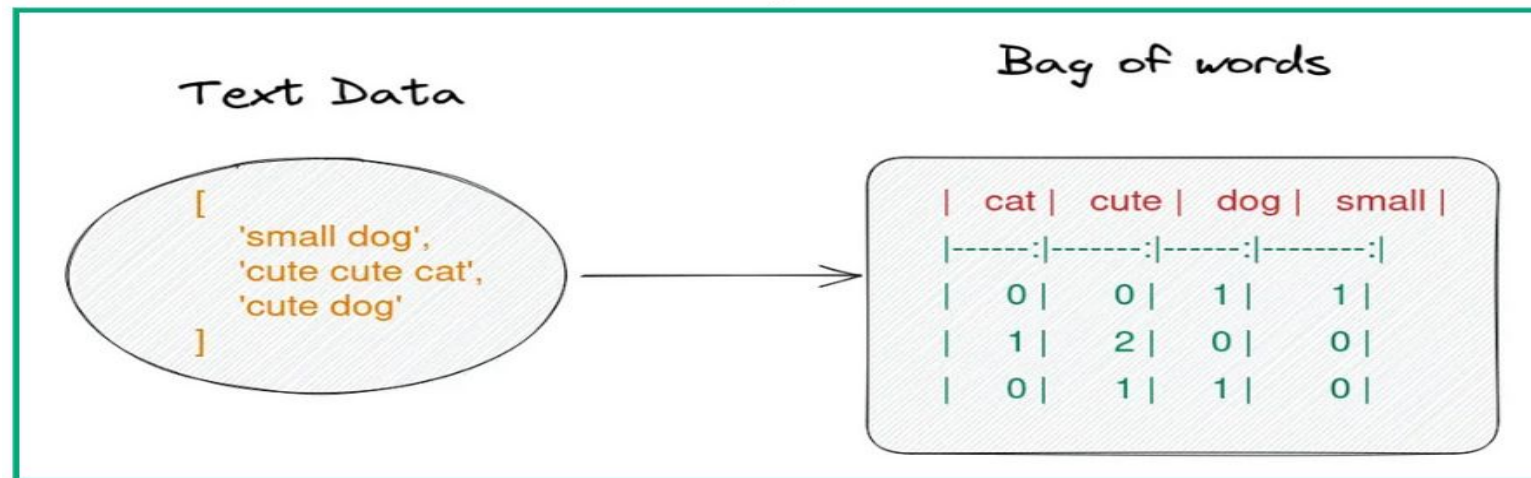
Feature Extraction

Bag of Words (BoW)

This approach transforms text into numerical form by counting word occurrences, while ignoring grammar and word order. It assumes that texts with similar meanings share common words.

Main steps:

- **Tokenization:** Break the text into individual words.
- **Vocabulary:** Build a list of all unique words from the dataset.
- **Vectorization:** Represent each document as a vector of word counts based on the vocabulary.



TF-IDF (Term Frequency - Inverse Document Frequency)

This method scores how important a word is in a document compared to all documents in the dataset. It reduces the impact of common words.

- **Term Frequency (TF):** How often a word appears in a single document.
- **Inverse Document Frequency (IDF):** How rare the word is across the entire dataset.
- The TF-IDF score is the product of TF and IDF:

$$TF(t, d) = \frac{\text{Number of term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t, D) = \log \left(\frac{\text{Total Number of documents}}{\text{Number of documents containing term } t} \right)$$

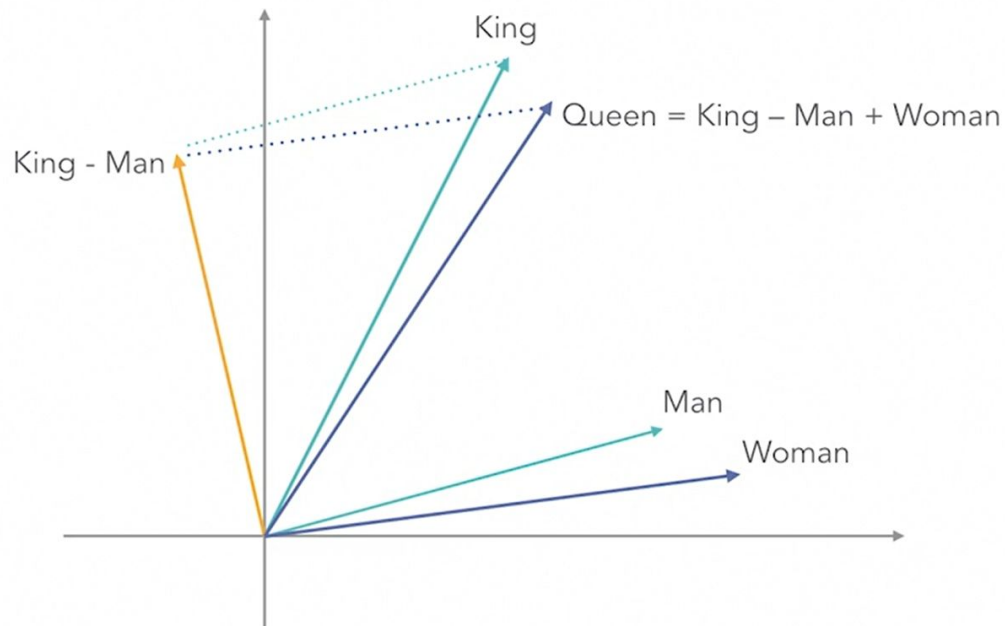
$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Word Embeddings (Word2Vec)

This method represents words in a dense vector space, capturing semantic relationships. Words with similar meanings are located close to each other in this vector space.

It is typically trained on large text corpora using neural networks. Here, we use the pre-trained 'Word2Vec' models developed by Google ('GoogleNews-vectors-negative300.bin')

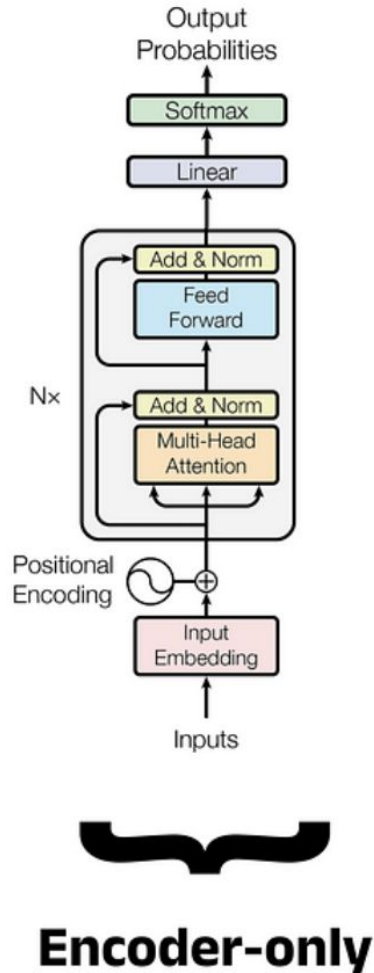
Twitter language is full of slang, abbreviations, and emojis. Word2Vec can learn these informal and evolving expressions, leading to accurate detection of hate tweets.



Example:

The vector of **King**
– The vector of **Man**
+ The vector of **Women**
= The vector of **Queen**

BERT (Bidirectional Encoder Representations from Transformers)



BERT is a pre-trained transformer-based model. It can understand the context of a word in a sentence, reading text in both direction (left-to-right and right-to-left).

- **Transformer:** it applies self-attention mechanism, allowing the model to weight the importance of different words in a sentence and capture long-range dependencies.
- **Positional Encoding:** it provides information about the position of each word in the sequence.
- **Encoder:** it processes the input text and generates a contextualized representation. The original transformer model consists of an encoder and a decoder, BERT only use the encoder.
 - 1) Multi-head self-attention mechanism: focus on different parts of the sentence simultaneously, capturing various aspects of the relationships between words.
 - 2) Feed-Forward Neural Networks (FFNN): introduces non-linearities and transforms the feature extracted by the self-attention mechanism, increasing the model's capacity to capture intricate relationships in the data.
 - 3) Add and normalize operations for layer normalization and residual connections: it stabilizes training and allows gradients to flow through the network more effectively.

Model Building & Training

- SVM
- Random Forest
- XGBClassifier
- CNN+LSTM
- Transformer (Fine-tune BERT)

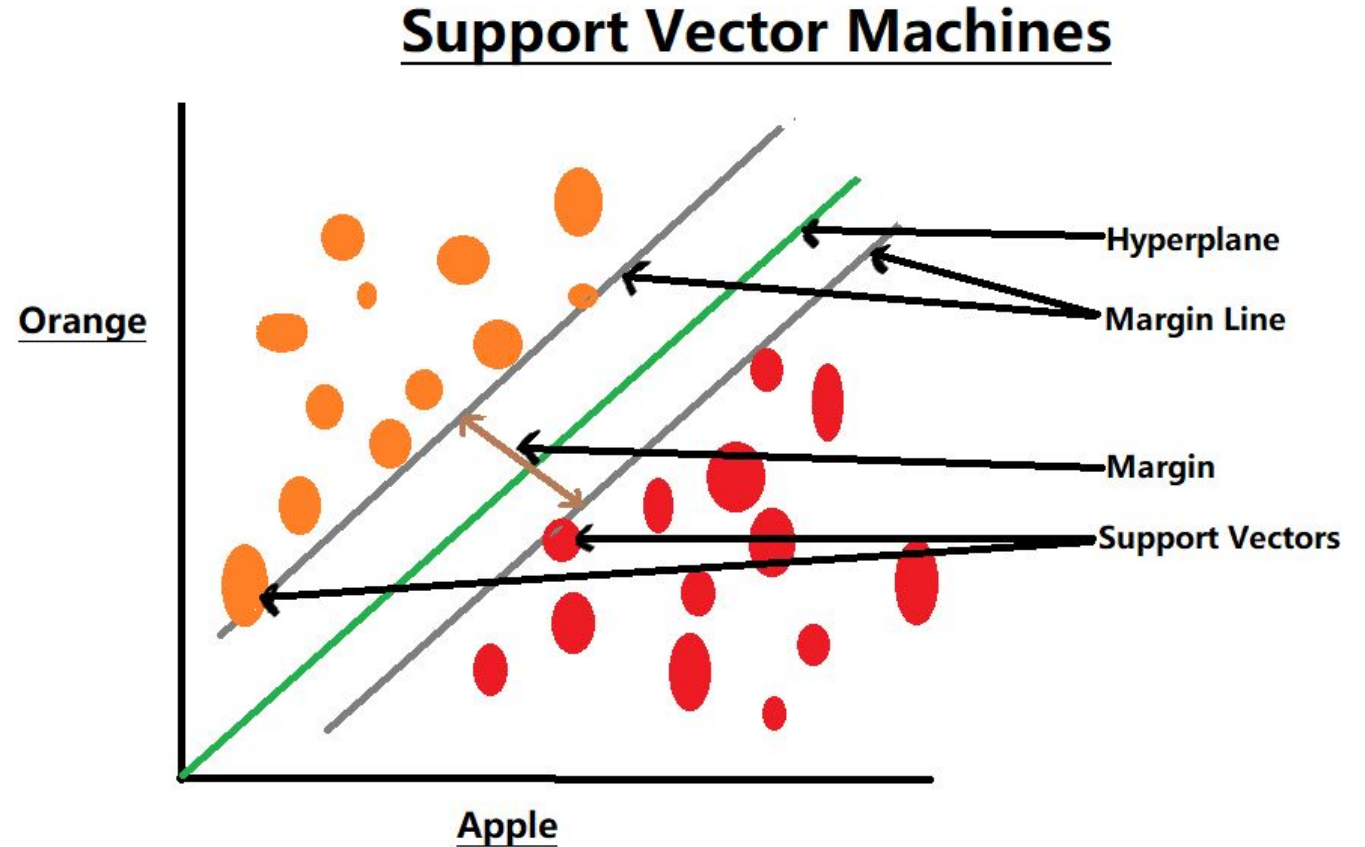


Data Glacier

Your Deep Learning Partner

SVM

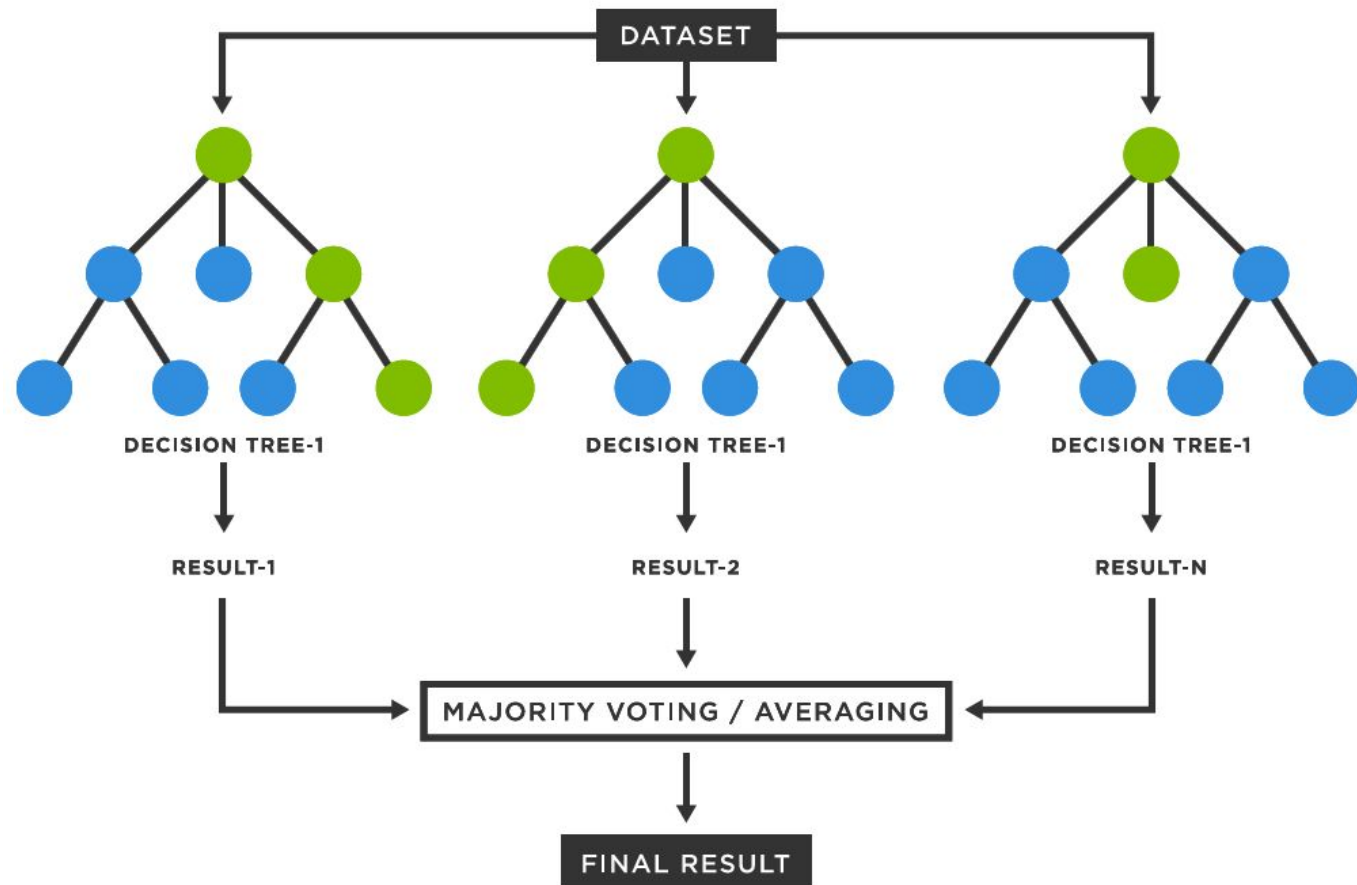
- Support Vector Machine is a powerful supervised learning algorithm that is especially effective in high-dimensional spaces like TF-IDF feature vectors.
- It works by finding the optimal hyperplane that maximizes the margin between different classes (hate and non-hate tweets).
- Due to its simplicity, robustness, and ability to handle sparse data, SVM serves as a strong linear baseline model in text classification tasks like hate speech detection.



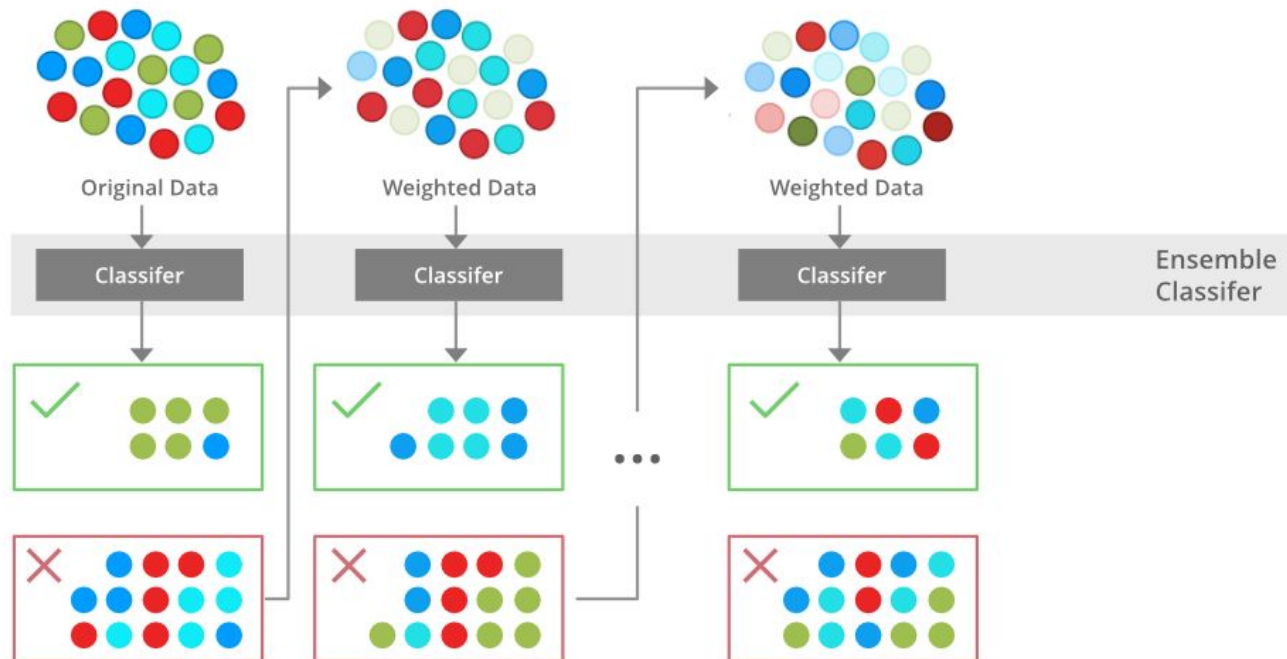
Random Forest

Random Forest model creates multiple decision trees using different subsets of the data (bootstrapping technique: sampling with replacement) and takes a majority vote (for classification task).

- At each split in a tree, a random subset of features is selected to find the best split, which introduces randomness and reduces overfitting.
- We add parameter `'n-estimators = 100'` when constructing the model. It means that the random forest will consist of 100 individual decision trees.



XGBClassifier



The XGBClassifier is part of the XGBoost (Extreme Gradient Boosting) library, which is gradient-boosted decision trees.

- This model sequentially builds trees where each tree tries to correct the errors of the previous trees.
- It optimizes an objective function using gradient descent.
- Different from random forest, **all features are considered at each split**, but the algorithm focuses on features that reduce the loss the most.
- We also add parameter ***n-estimators = 100*** in model construction.
- Each tree is trained on a random subset of the training data, it contributes to the overall robustness and performance of the model.

CNN + LSTM Model Architecture

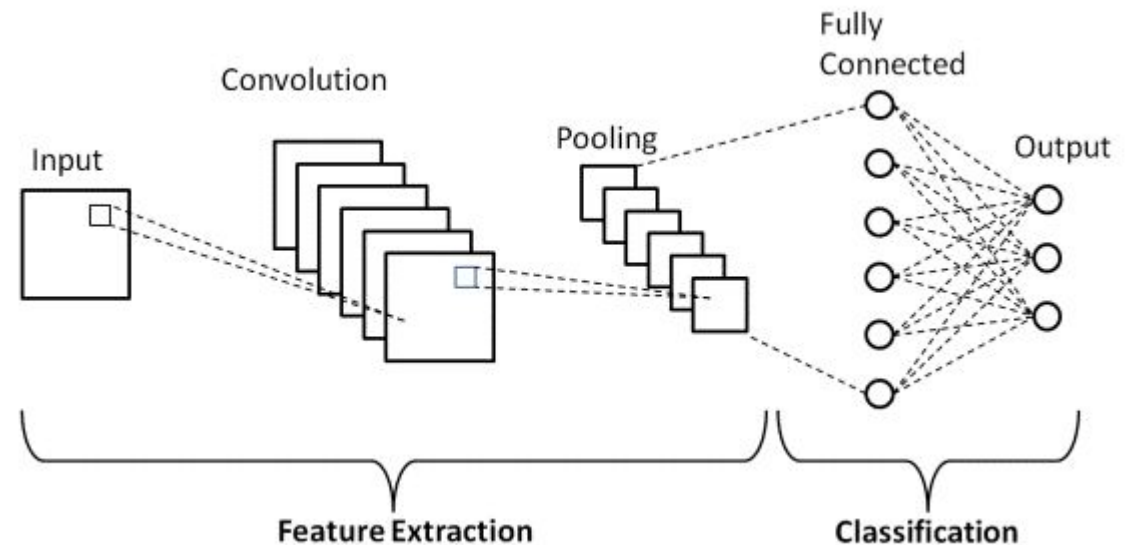
The CNN + LSTM hybrid architecture leverages the strengths of both convolutional and recurrent networks.

- **CNN Layer:** Captures local n-gram features (e.g., offensive phrases or repeated patterns) from the word embeddings.
- **LSTM Layer:** Learns the sequential flow of language to understand contextual patterns across time steps.
- **Dropout & Dense Layers:** Help in regularization and final binary classification of hate vs non-hate content.

CNN (Convolutional Neural Network)

The **Convolutional Neural Network (CNN)** is highly effective for detecting local patterns in text such as phrases or specific word combinations often present in hate speech.

- **Word Embeddings Input:** Each tweet is converted into a fixed-length vector using GloVe embeddings.
- **Conv1D Layer:** Applies filters (kernels) over the sequence to extract local features (e.g., "kill them", "go back").
- **Max Pooling Layer:** Reduces dimensionality and retains the most significant features.
- **Dense Layers:** Final classification layer to predict whether the tweet is hate or non-hate.



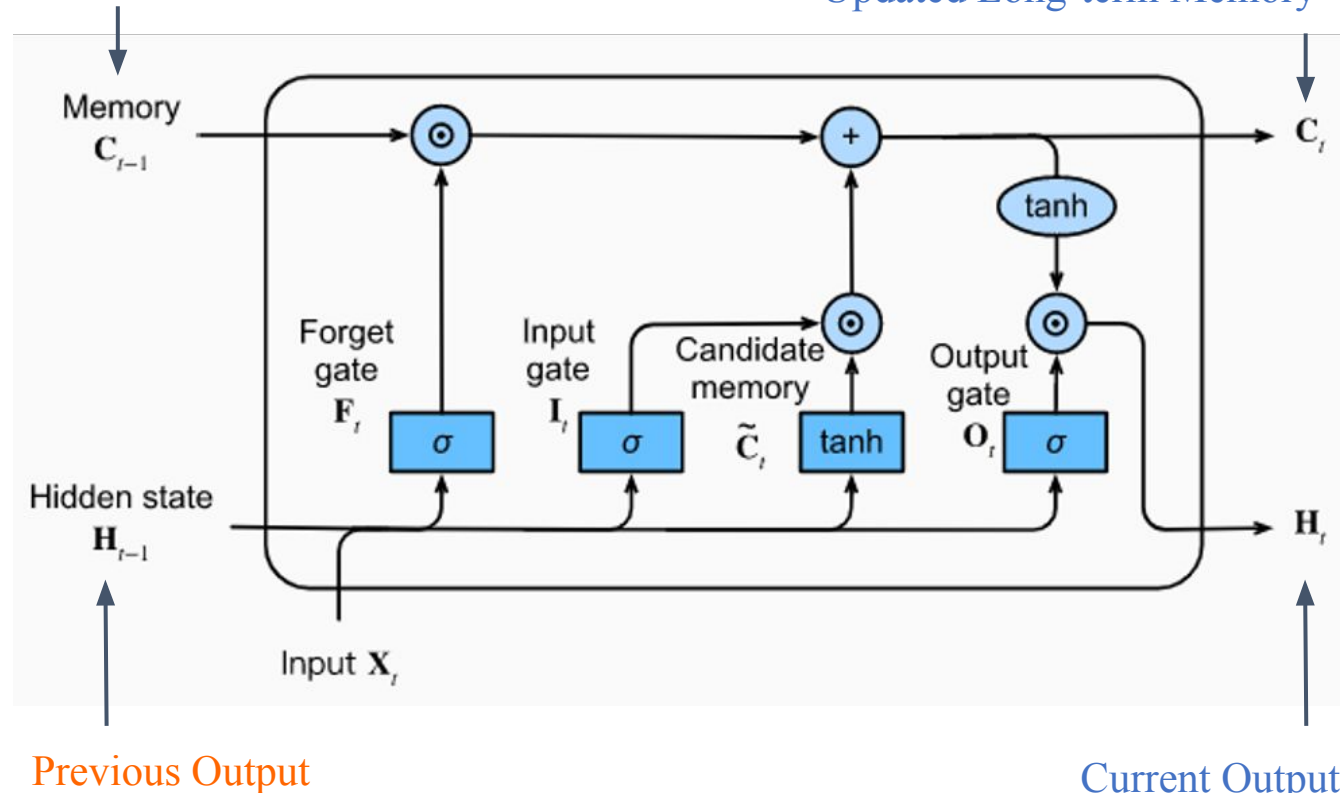
LSTM (Long Short-Term Memory)

LSTM models is a neural network that can capture and understand the context and dependencies within text sequences.

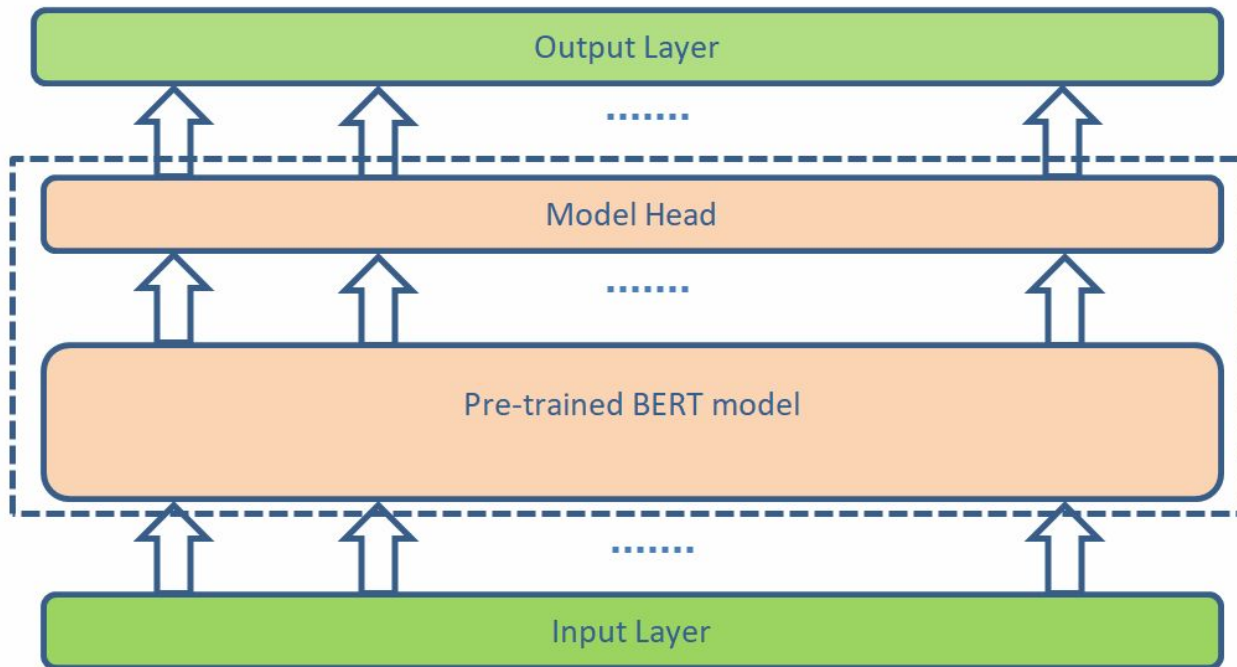
- It is introduced to address the vanishing gradient problem of RNNs (Recurrent Neural Network).
- LSTM networks use memory cell structure that can maintain information over long periods.
- It uses gating mechanisms to control the flow of information:
 - 1) **Input gate**: add/update new information
 - 2) **Forget gate**: forget irrelevant information
 - 3) **Output gate**: pass the updated information

Long-term Memory

Updated Long-term Memory



Transformer (Fine-tune BERT)



Transformer (BERT) model uses self-attention mechanisms to understand the context of a word based on its surrounding words.

- Here we fine-tune the pre-trained BERT model on our specific tweet speech dataset and hate speech detection task.
- We **update weights** and of the pre-trained BERT model.
- We use **`TFBertForSequenceClassification`** class when loading the model to adapt the BERT model by adding a classification head (usually a linear layer) on top of the BERT encoder.

Train Test Split

For SVM, Random Forest and XGBClassifier:

- Training set: 80%, Test set: 20%

For CNN+LSTM:

- Training set: 80% (In each epoch, randomly select 20% of training set as Validation set), Test set 20%
- Training stops at 10th epoch

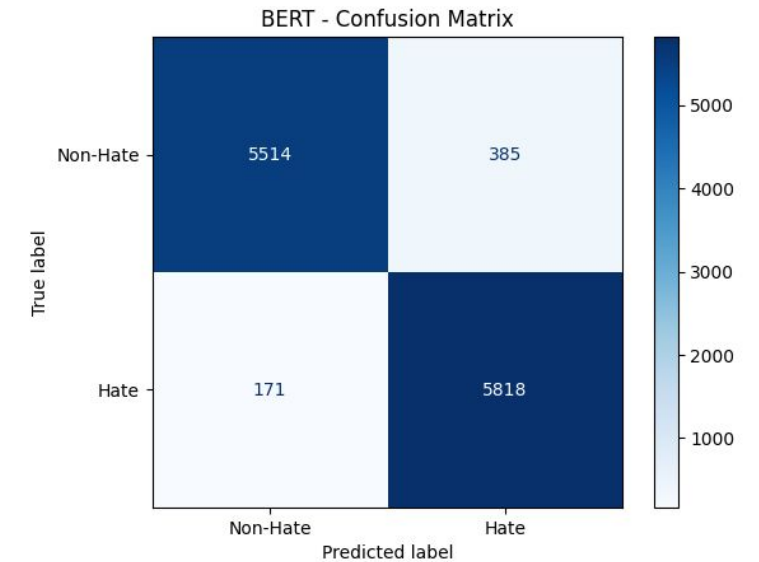
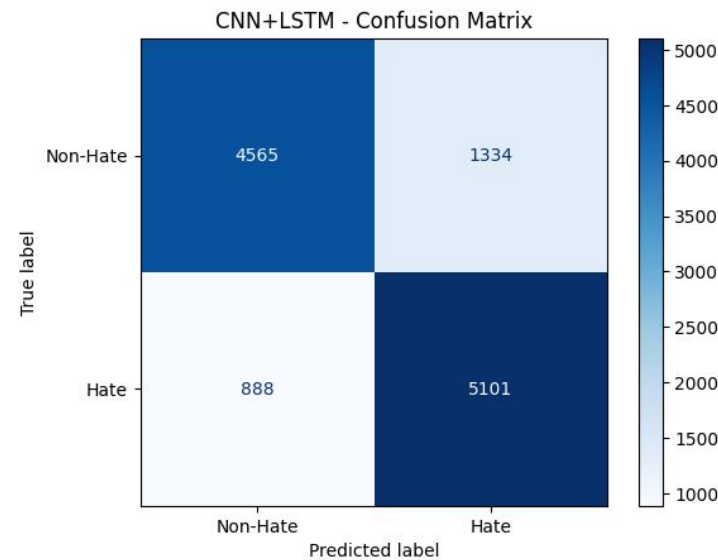
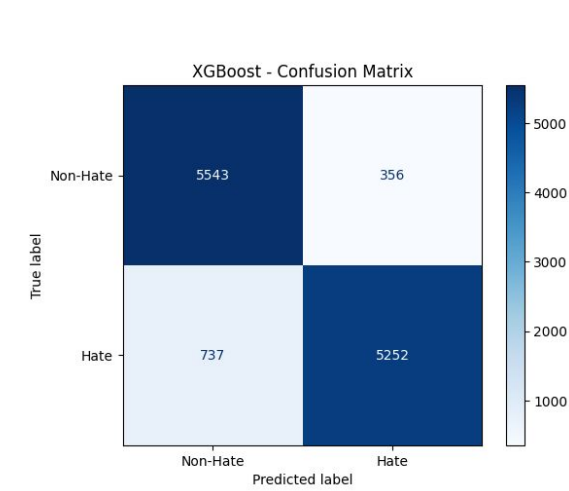
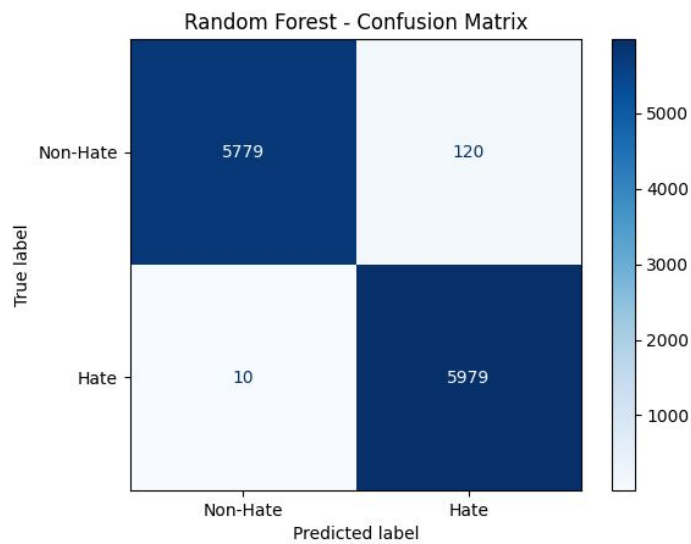
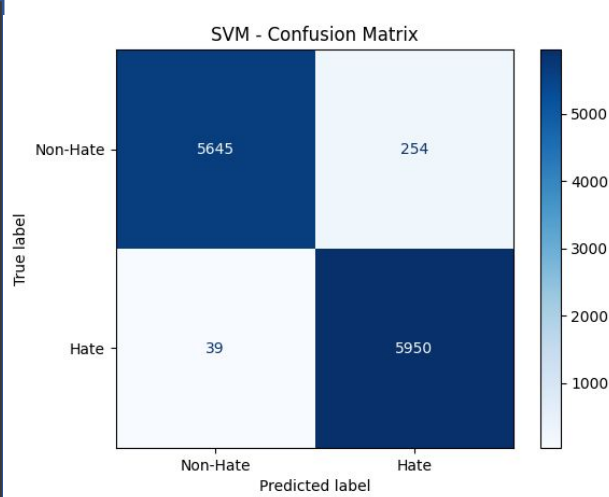
For Transformer (Fine-tune BERT):

- Training set: 60%, Validation Set 20%, Test set: 20%
- Learning rate = 0.00005
- Training stops at 5th epoch

- Confusion Matrix
- Accuracy, F1-Score, ROC-AUC Score
- ROC Curve

Model Performance Evaluation

Confusion Matrix



The **Random Forest** performs best on predicting **non-hate tweets**, & **hate tweets**.



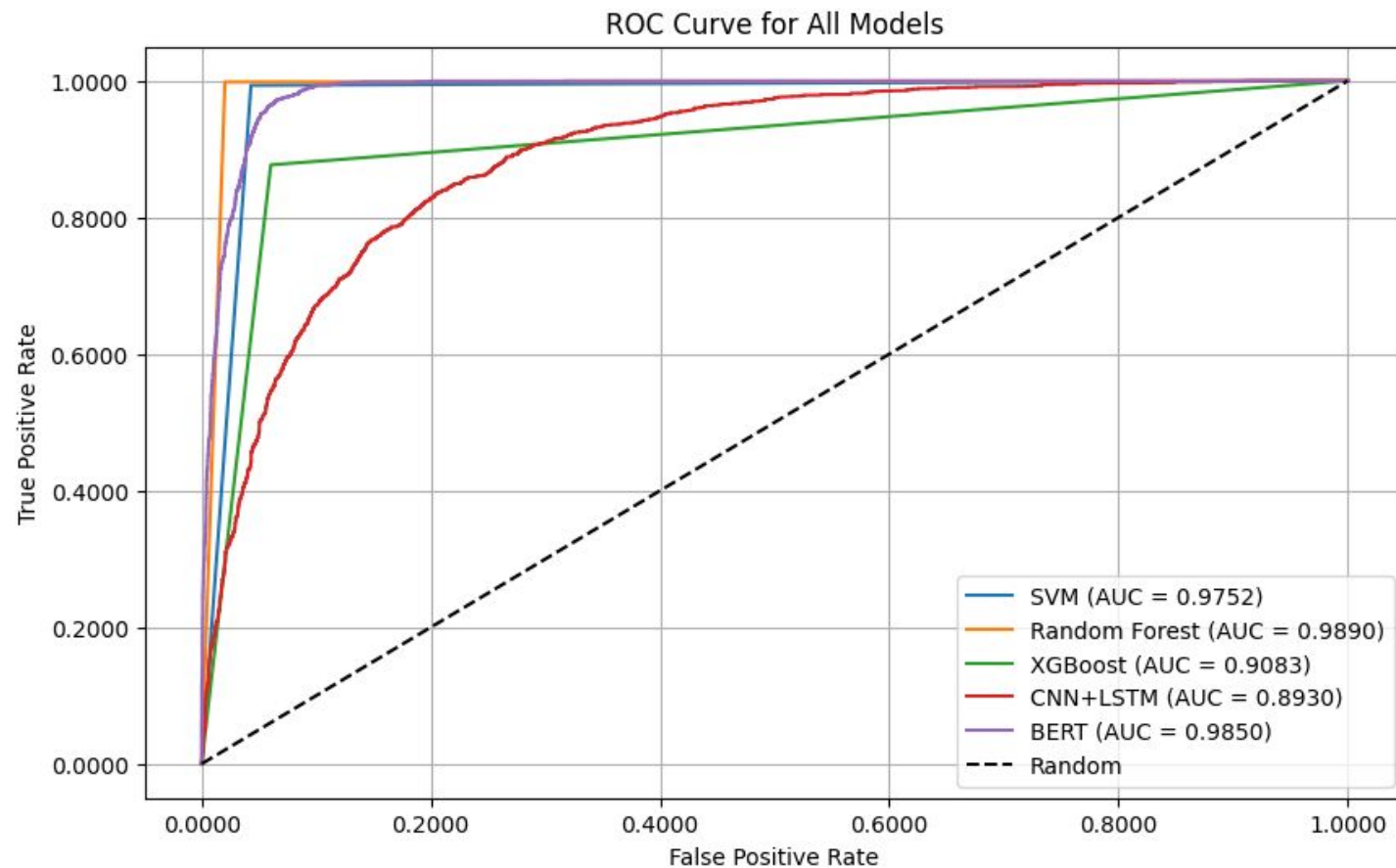
Data Glacier

Your Deep Learning Partner

Accuracy, F1-Score, ROC-AUC Score

Model	Accuracy	F1-Score	ROC-AUC Score
SVM	0.98	0.98	0.9752
Random Forest	0.99	0.99	0.9890
XGBClassifier	0.91	0.91	0.9083
CNN+LSTM	0.81	0.81	0.8930
Transformer (Fine-tune BERT)	0.95	0.95	0.9850

- The Random Forest has the highest accuracy (99%), while the CNN+LSTM has the lowest accuracy (81%).
- The Random Forest performs best on predicting both hate and non-hate tweets.
- The Random Forest model has the highest ROC-AUC score, which means that it has the best overall performance and discrimination capability of all five models we used.



ROC Curve

The Random Forest model has the highest AUC (area under the curve). It is the orange line, which is overlapped with purple line and blue line. This model has the best ability to distinguish between hate and non-hate tweets across different threshold settings.



Application Design

- Deploy the ML model on Flask to create a web application
- Running procedure

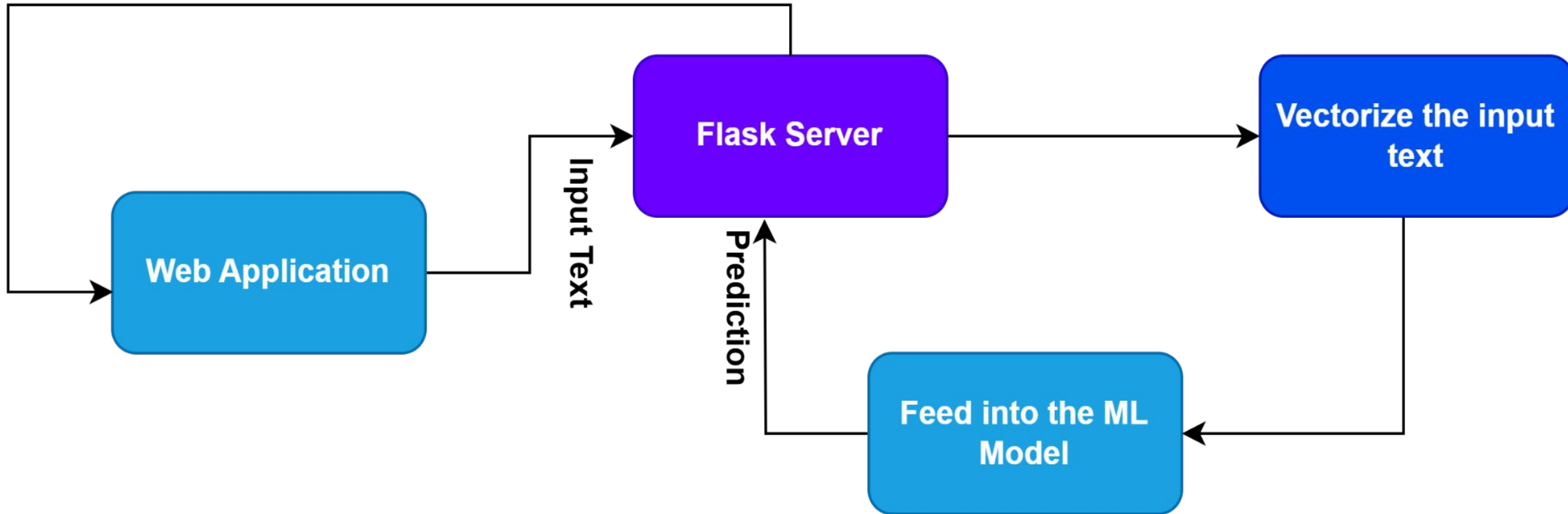


Data Glacier

Your Deep Learning Partner

Deploy the ML model on Flask to create a web application

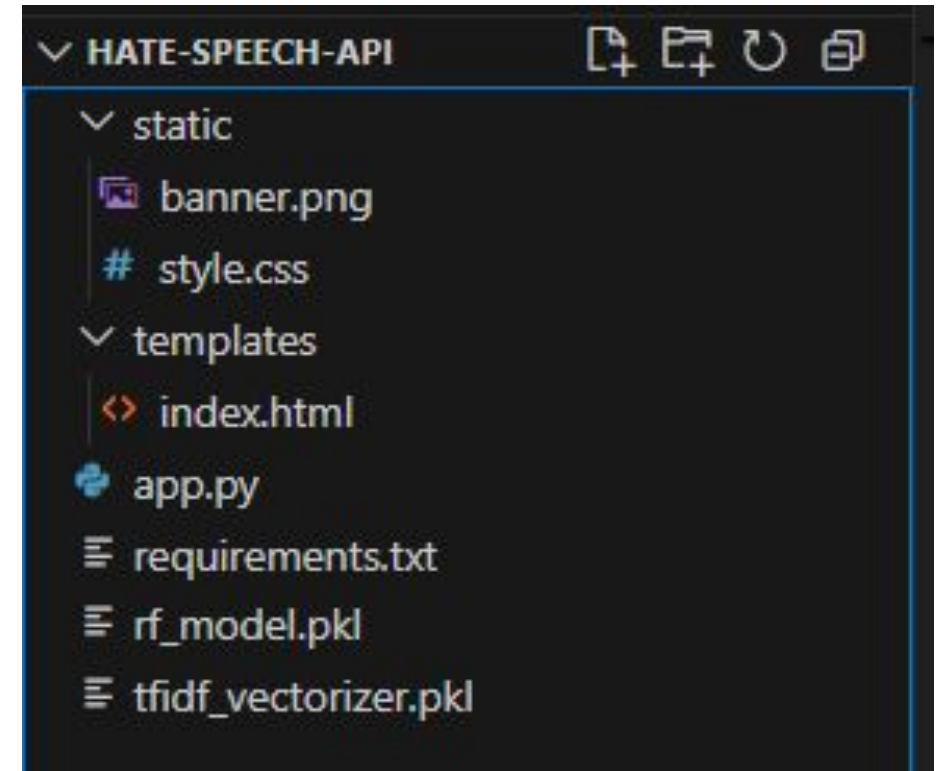
Workflow:



Deploy the ML model on Flask to create a web application

Application Files:

- ***banner.png***: image shown on the website
- ***style.css***: the style of our web design
- ***index.html***: illustrate all the components on the front-end website
- ***app.py***: the Flask application code
- ***requirements.txt***: lists all the package needed to run the Flask web application
- ***tfidf_vectorizer.pkl***: the TF-IDF vectorizer
- ***rf_model.pkl***: our pre-trained Random Forest model



Run the *app.py* file on the terminal, and click the link of website in the output:

<https://hate-speech-api-qyjg.onrender.com/>



Twitter Hate Speech Detector

Finding Hate Tweet Here

Paste a tweet here...

Analyze Tweet

© Mehedi Hasan Shakil, 2025. All Rights Reserved.

Running Procedure: Website Overview



Data Glacier

Running Procedure: Enter Input Text

Enter a speech/tweet.

Example: I support white supremacy, white people should run the world



The screenshot shows a web application titled "Twitter Hate Speech Detector". It features a dark header with a white "X" logo. The main content area is light blue and contains a white card with the heading "Finding Hate Tweet Here". Inside the card is a text input field containing the example text "I support white supremacy, white people should rule the world" and a blue "Analyze Tweet" button. The footer is dark and contains the copyright notice "© Mehedi Hasan Shakil, 2025. All Rights Reserved."



Data Glacier

Your Deep Learning Partner

After clicking the 'Submit' button, the speech/tweet is first passed through the TF-IDF vectorizer, then fitted into our ML model, and the classifier will output a result and display on the website.



The screenshot shows a web application titled "Twitter Hate Speech Detector". It features a black header with a white "X" logo. The main content area is light blue and contains a white box with the heading "Finding Hate Tweet Here". Inside this box is a text input field with the placeholder "Paste a tweet here...". Below the input field is a blue button labeled "Analyze Tweet". At the bottom of the white box, the prediction is displayed as "Prediction: Hate Speech" with a red "X" icon. The footer is black and contains the copyright notice "© Mehedi Hasan Shakil, 2025. All Rights Reserved."

**Running Procedure:
Predict the Result**

Conclusion

- We developed a **robust hate speech detection system** using traditional machine learning and deep learning models.
- The **Random Forest classifier with TF-IDF features** achieved the best balance between accuracy, precision, and recall.
- A **web application** was deployed using Flask and Render to allow real-time tweet classification.
- This project demonstrates how **NLP and ML techniques** can effectively combat toxic content on social platforms.



Data Glacier

Your Deep Learning Partner

Reference

- [1] Dataset: <https://www.kaggle.com/datasets/vkrahul/twitter-hate-speech/code>

- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780. DOI: 10.1162/neco.1997.9.8.1735

- [3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Thank You



Data Glacier

Your Deep Learning Partner